

Assignment 6: Automation in SRE, Capacity Planning and Load Testing

Individual Project Report

Student: _____

Course: Site Reliability Engineering

Date: May 2026

1. Introduction

This report demonstrates automation in Site Reliability Engineering for a growing e-commerce API. The work includes automated log extraction, monthly traffic growth calculation, six-month traffic forecasting, Kubernetes Horizontal Pod Autoscaler configuration, traffic spike simulation using Locust, and ROI/incident analysis.

2. Objective and Scenario

The objective is to understand how automation helps SRE teams predict traffic growth, prepare infrastructure capacity, and keep the service stable during unexpected load spikes. The scenario is a rapidly growing e-commerce API service.

3. Predictive Analysis: Python Log Extraction and Forecasting

The Python script `analysis/traffic_forecast.py` automatically extracts access log records, groups requests by month, calculates month-to-month growth, and forecasts the next six months using average monthly growth.

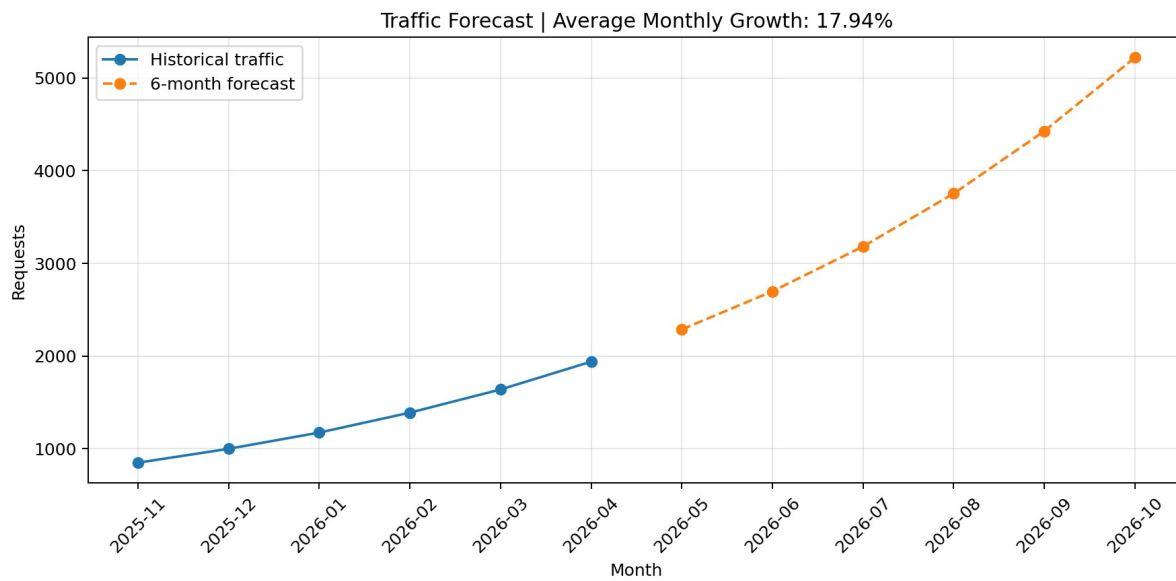
3.1 Forecasting Logic

The formula used for each future month is: $\text{forecast} = \text{last_actual_traffic} * (1 + \text{average_growth})^n$, where n is the number of months after the latest actual month. In this dataset, the calculated average monthly growth is 17.94%.

3.2 Forecast Output

Month	Actual Requests	Forecast Requests
2025-11	850	
2025-12	1000	
2026-01	1175	
2026-02	1390	
2026-03	1640	
2026-04	1940	
2026-05		2288
2026-06		2699
2026-07		3183
2026-08		3754
2026-09		4428
2026-10		5222

Figure 1. Forecasted traffic output generated by Python.



4. Infrastructure Scaling with Kubernetes HPA

The e-commerce API is deployed as a Kubernetes Deployment. CPU requests and limits are configured because HPA CPU utilization is calculated relative to requested CPU. The Horizontal Pod Autoscaler targets the Deployment and keeps average CPU utilization around 80% by increasing or decreasing the number of pods.

4.1 HPA Configuration

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ecommerce-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ecommerce-api
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 80
```

Figure 2. HPA status after applying hpa.yaml.

INSERT SCREENSHOT HERE

kubectl get hpa with 80% CPU target

5. Load Testing Simulation

Locust is used to simulate traffic spikes. The test sends frequent requests to /cpu?ms=250, an intentionally CPU-heavy endpoint. This increases pod CPU usage and allows HPA to demonstrate automatic scaling.

5.1 Load Test Command

```
locust -f load_test/locustfile.py --host http://localhost:8080 --users 150 --spawn-rate 20 --run-time 3m --headless
```

Figure 3. Locust load testing output.

INSERT SCREENSHOT HERE

Locust load test output / web UI

Figure 4. Kubernetes pods after HPA scaling.

INSERT SCREENSHOT HERE

kubectl get pods showing scaled replicas

6. Real-World Incidents Preventable by Automation

Traffic spike overload: Without autoscaling, sudden campaigns or seasonal sales can overload a small number of pods. HPA prevents this by increasing replicas automatically when CPU usage rises.

Slow manual capacity response: If engineers manually add capacity only after complaints start, users experience slow responses or failures. Automated monitoring and scaling reduce response time.

Late detection of growth trends: If teams do not analyze logs, monthly traffic growth can be missed until the system is already unstable. Forecasting gives time to plan resources before incidents happen.

7. ROI Analysis of SRE Automation

SRE automation creates ROI by reducing downtime, manual engineering work, and over-provisioning costs. Forecasting helps predict future traffic and avoid last-minute infrastructure changes. HPA reduces wasted resources because the application can run with fewer pods during normal load and scale up only during traffic spikes. Load testing gives evidence that the system can survive higher demand before real users are affected.

Example ROI logic: if automation prevents even one one-hour outage during a high-traffic sale, the saved revenue and reputation can be higher than the cost of implementing scripts, monitoring, and autoscaling. Automation also improves engineer productivity because repeated tasks such as log extraction, forecast calculation, and scaling checks are handled by scripts and Kubernetes.

8. Conclusion

This project shows how SRE automation connects prediction, scaling, and validation. Python forecasting identifies expected future load, Kubernetes HPA provides automatic scaling at an 80% CPU threshold, and Locust proves scaling behavior under simulated traffic. Together, these practices make the service more reliable and reduce operational risk.

9. References

Kubernetes Documentation: Horizontal Pod Autoscaling.

Kubernetes Documentation: Horizontal Pod Autoscaler Walkthrough.

Locust Documentation: Quickstart and headless load testing.

Kubernetes Metrics Server Documentation.